

Proyecto: Simulación de Eventos Discretos - Modelo de Inventario (s, S)

Abel Ponce González Grupo: C-311 Telegram: @albeceonp

14 de abril de 2025

S1 Introducción

Descripción del problema

- En una tienda que almacena solo un tipo de producto, el cual vende a un precio unitario r .
- Los clientes que demandan este producto aparecen según un proceso de Poisson con una tasa λ .
- La cantidad de demanda por cada uno es una variable aleatoria con distribución G .
- Para satisfacer la demanda, el vendedor debe mantener una cantidad del producto disponible.
- Cuando el inventario es bajo solicita unidades adicionales al distribuidor.
- El vendedor utiliza una política de pedidos denominada (s, S) .
- Si el nivel actual de inventario es x y no hay ningún pedido pendiente, si $x < s$, se solicita la cantidad $S - x$.
- El coste de solicitar y unidades del producto, es una función específica $C(y)$.
- La entrega del producto tarda L unidades, realizándose el pago en el momento de la entrega.
- La tienda paga un coste de mantenimiento de inventario de h por unidad de artículo por unidad de tiempo.
- Siempre que un cliente demande más productos del disponible se vende la cantidad disponible, se vende la cantidad existente y la tienda pierde el resto del pedido.

Objetivos y metas

- Determinar políticas óptimas (s, S) que maximicen el beneficio.
- Analizar el impacto de distribuciones de demanda (Normal vs. Exponencial).
- Cuantificar incertidumbre mediante análisis bootstrap.

Variables del sistema

- t : Variable de tiempo.
- x : Cantidad de inventario a la mano.
- y : Cantidad solicitada.
- s : Punto de reorden.
- S : Nivel máximo de inventario.
- λ : Tasa de llegada de clientes (Poisson).
- h : Costo de mantenimiento por unidad/tiempo.

- C : La cantidad total de costos de los pedidos hasta t .
- R : La cantidad total de ingresos obtenidos hasta t .
- H : La cantidad total de costos de mantenimiento de inventario hasta t .

Los eventos del sistema serán la llegada de un cliente o de un pedido. Los tiempos de evento son:

- t_0 : El tiempo de llegada del siguiente cliente.
- t_1 : El tiempo de entrega de un pedido. Si no hay pedido pendiente, entonces $t_1 = \infty$

S2 Detalles de Implementación

EL sistema fue modelado en Python de forma tal que la llegada de los clientes fue determinada por una variable aleatoria que distribuye **Poisson**($\lambda = 3$). En cuanto la demanda de los clientes se analizaron los casos de cuando distribuía normal y exponencial. La función utilizada para el costo del pedido para una cantidad y de unidades fue: $C(y) = 50 + 2 * y$ donde establecemos un coste fijo de 50 unidades monetarias por realizar cualquier pedido, esta función penaliza pedir en exceso, ya que incrementa el costo total linealmente. Para una simulación se consideraron 10000 unidades de tiempo.

Implementación de modelo

```
def Simular(self):
    """Ejecuta la simulación hasta el tiempo T."""
    while self.t < self.T:
        if self.t0 < self.t1:
            self.manejar_llegada_de_clientes()
        else:
            self.manejar_pedidos_de_entrega()
    return self.resultados()
```

```
def manejar_llegada_de_clientes(self):
    """Procesa la llegada de un cliente (Caso 1)."""
    # Actualizar costos de mantenimiento
    self.H += (self.t0 - self.t) * self.x * self.h
    self.t = self.t0

    # Generar demanda y vender
    D = self.G()          # Demanda del cliente
    w = min(D, self.x)    # Unidades vendidas
    self.R += w * self.r   # Actualizar ingresos
    self.x -= w           # Reducir inventario

    # Hacer pedido si es necesario (y no hay pedidos pendientes)
    if self.x < self.s and self.y == 0:
        self.y = self.S - self.x
        self.t1 = self.t + self.L
        self.C += self.c(self.y) # Añadir costo del pedido

    # Programar próxima llegada de cliente
    self.t0 = self.generar_proximo_cliente()
```

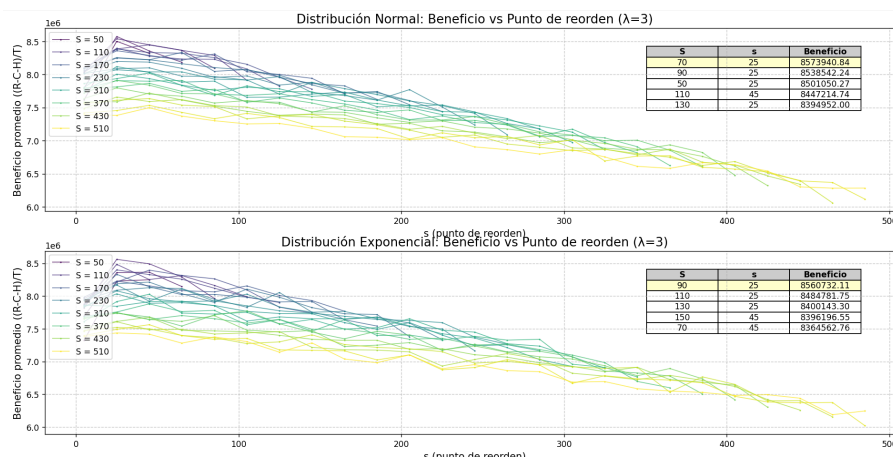
```
def manejar_pedidos_de_entrega(self):
    """Procesa la entrega de un pedido (Caso 2)."""
    # Actualizar costos de mantenimiento
    self.H += (self.t1 - self.t) * self.x * self.h
    self.t = self.t1

    # Recibir pedido y reiniciar variables
    self.x += self.y
    self.y = 0
    self.t1 = float('inf')
```

S3 Resultados y Experimentos

Hallazgos principales

Luego de hacer simulaciones para muchas combinaciones de s y S se obtuvo el siguiente gráfico donde se muestran los valores óptimos de los mismos para distribución de demanda normal y exponencial.



A partir de los valores óptimos de s y S que maximizan el beneficio por unidades de tiempo se escogieron los dos mejores para hacer bootstrapping donde se estiman los intervalos de confianza del 95 % para estimar la robustez de las políticas (s, S) .

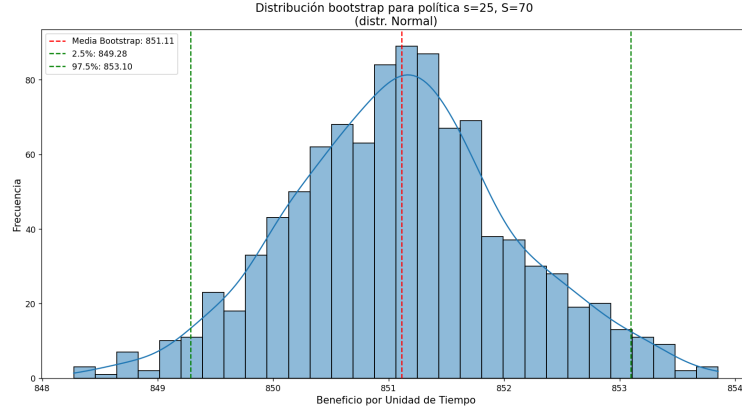


Figura 1: Distribución bootstrap para política (25,70)

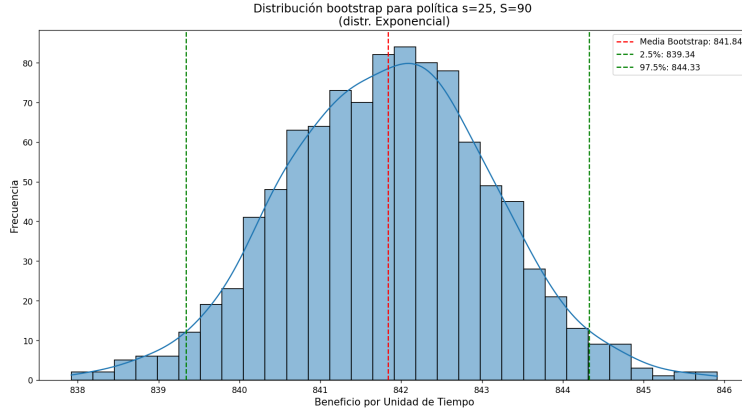


Figura 2: Distribución bootstrap para política (25,90)

- Bajo demanda Normal: Política óptima (25, 70) con beneficio promedio de 851,11 (IC 95 %: [849,28, 853,10]).
- Bajo demanda Exponencial: Política (25, 90) alcanza 841,84 (IC 95 %: [839,34, 844,33]).

Cuadro 1: Comparación de políticas		
Distribución	Políticas	Resultado
Normal	(25, 70) vs (25, 90)	$p > 0,05$ (NS)
Exponencial	(25, 90) vs (25, 70)	$p > 0,05$ (NS)

Se ejecutaron 30 réplicas por política hasta $T = 10,000$ unidades de tiempo, alcanzando estabilidad en los resultados a las 5,000 unidades.

S4 Modelo Matemático

Función de Costos y Beneficio

- Costo total de pedidos:

$$C = \sum_{k=1}^N (50 + 2y_k) \quad (\text{costo fijo} + \text{variable})$$

- **Costo de mantenimiento:**

$$H = h \int_0^T x(t) dt$$

- **Artículos que obtiene el cliente**

$$w = \min(D, x) \quad (\text{La cantidad de pedido que se puede surtir})$$

- **Ingresos:**

$$R = r \sum_{i=1}^M w_i \quad (M = \text{número de clientes atendidos})$$

- **Beneficio neto:**

$$\Pi = R - C - H$$

Supuestos clave

- Tiempos de entrega fijos ($L = 1,0$).
- Costo de mantenimiento por artículo por unidad de tiempo ($h = 0,5$).
- No se permiten pedidos simultáneos.
- Función de costo $50 + 2y$.
- Demandas no satisfechas se pierden.

S5 Conclusiones

- La política (25, 70) minimiza costos bajo demanda estable (Normal).
- La alta variabilidad de la demanda Exponencial requiere mayores niveles S .
- Los intervalos de confianza superpuestos sugieren múltiples políticas viables.
- Las diferencias de beneficio por unidad de tiempo para las políticas (s,S) óptimas escogidas no es significativa.